Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3677052.3698613

PDF Download
3677052.3698613.pdf
29 March 2026
Total Citations: 1
Total Downloads: 678

Published: 14 November 2024

Citation in BibTeX format

ICAIF '24: 5th ACM International Conference on AI in Finance
November 14 - 17, 2024
NY, Brooklyn, USA

RESEARCH-ARTICLE

RiskMiner: Discovering Formulaic Alphas via Risk Seeking Monte Carlo Tree Search

TAO REN, Peking University, Beijing, China

RUIHAN ZHOU, Peking University, Beijing, China

JINYANG JIANG, Peking University, Beijing, China

JIAFENG LIANG, Harbin Institute of Technology, Harbin, Heilongjiang, China

QINGHAO WANG, Peking University, Beijing, China

YIJIE PENG, Peking University, Beijing, China

Open Access Support provided by:

Peking University

Harbin Institute of Technology

ICAIF '24: Proceedings of the 5th ACM International Conference on AI in Finance (November 2024)
https://doi.org/10.1145/3677052.3698613
ISBN: 9798400710810

RiskMiner: Discovering Formulaic Alphas via Risk Seeking Monte Carlo Tree Search

Tao Ren
peking university
CN
rtkenny@stu.pku.edu.cn

Ruihan Zhou
Peking university
CN
rhzhou@stu.pku.edu.cn

Jinyang Jiang
Peking University
CN
jinyang.jiang@stu.pku.edu.cn

Jiafeng Liang
Harbin Institute of Technology
CN
jfliang@ir.hit.edu.cn

Qinghao Wang
Peking University
CN
qinghw@pku.edu.cn

Yijie Peng*
Peking University
CN
pengyijie@pku.edu.cn

Abstract

The formulaic alphas are mathematical formulas that transform raw stock data into indicated signals. In the industry, a collection of formulaic alphas is combined to enhance modeling accuracy. Existing alpha mining methods only employ the neural network agent, resulting in over-exploitation. Moreover, they didn't consider the correlation between alphas in the collection, limiting the synergistic performance. To address these problems, we propose a novel alpha mining framework, which formulates the alpha mining problems as a reward-dense Markov Decision Process (MDP) and solves the MDP by the risk-seeking Monte Carlo Tree Search (MCTS). The MCTS, designed for exploration, utilizes the structural information of discrete solution space; the risk-seeking policy, designed for exploitation, optimizes the best-case performance rather than average outcomes. Comprehensive experiments are conducted to demonstrate the efficiency of our framework. Our method outperforms all state-of-the-art benchmarks on real-world stock market data. The trading simulation shows that our alphas achieve the most profitable results under a realistic setting.

CCS Concepts

• **Computing methodologies** → **Reinforcement learning**; *Search methodologies*; • **Applied computing** → *Economics*.

Keywords

Computational Finance, Stock Trend Forecasting, Reinforcement Learning, Search Algorithm

ACM Reference Format:
Tao Ren, Ruihan Zhou, Jinyang Jiang, Jiafeng Liang, Qinghao Wang, and Yijie Peng. 2024. RiskMiner: Discovering Formulaic Alphas via Risk Seeking Monte Carlo Tree Search. In *5th ACM International Conference on AI in Finance (ICAIF '24), November 14–17, 2024, Brooklyn, NY, USA.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3677052.3698613

*Corresponding authors.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICAIF '24, November 14–17, 2024, Brooklyn, NY, USA
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1081-0/24/11
https://doi.org/10.1145/3677052.3698613

1 Introduction

Alpha factors or alphas transform raw stock data into indicated signals for future stock trends. There are two types of alphas: formulaic alphas[3, 8, 20, 22] and machine-learning alphas[7, 16, 17, 21, 23]. The formulaic alphas[14] are mathematical formulas, performing feature engineering on the stock data. The formulaic alphas are used either as trading signals or as input for machine-learning models. The machine learning alphas refer to machine learning models designed to produce trading signals[4]. Even though they usually are more predictive than formulaic alphas, the black-box nature renders the lack of interpretability.

Recently, researchers have designed various frameworks [3, 20, 22] to generate formulaic alphas automatically. Most of the alpha generation methods are based on genetic programming (GP) [3, 8, 22]. By performing biological operations on the expression tree, new alphas are generated based on existing ones. Alphagen[20] uses Reinforcement Learning (RL) to generate alphas. They formulate the mining pipeline as a Markov decision process (MDP) and use proximal policy optimization (PPO) [12] to solve it. Even though they have achieved better performance than GP, their pipeline has critical limitations. First, the reward-sparse nature of the MDP makes the learning highly non-stationary, as it provides limited feedback. Second, the neural network agent of PPO usually overly exploits local optimums of short expressions and fails to explore the huge solution space. Third, the alphagen only considers the average performance whereas the best-case performance should be the focus when mining alphas. To overcome these flaws, we introduce RiskMiner, a novel alpha mining framework addressing all the limitations.

Figure 1 illustrates the main idea of our framework. The Monte Carlo Tree Search (MCTS) serves as a sampler providing training data for risk-seeking policy optimization. Meanwhile, the trained risk-seeking policy network will be used in the MCTS. The two modules operate alternately as an alpha mining agent.

To solve the first limitation, our approach involves a reward-dense MDP, which is structured with both intermediate and terminal rewards, thereby stabilizing the learning. The mining agent can discover "excellent and different" synergistic alphas by solving the reward-dense MDP.

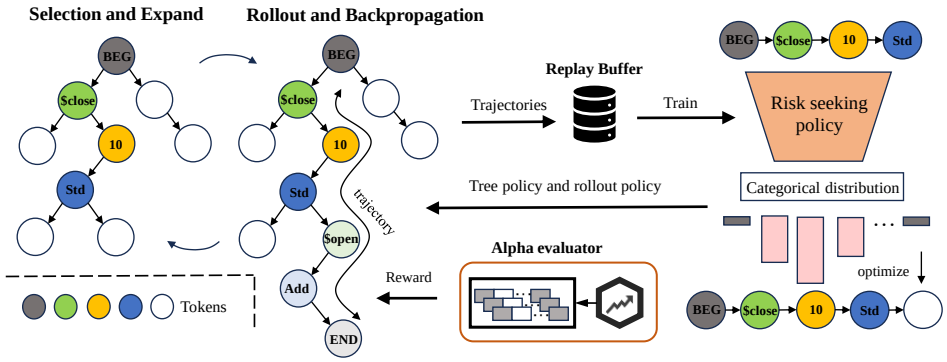


Figure 1: Algorithm framework diagram. The alpha evaluator provides reward signals for the reward-dense MDP. On the left side, the MCTS serves as a sampler to sample trajectories from the MDP. On the right side, a risk-seeking policy is trained and the policy will be used as the tree policy and rollout policy in the MCTS.

To solve the second limitation, we employ the MCTS, a potent tool for discrete planning problems, to search formulas based on risk-seeking policies. MCTS plays a pivotal role in the champion-defeating Go algorithm[13]. There has been a series of research applying the MCTS framework to solve other real-world problems including discovering faster sorting [9] and matrix multiplication algorithms [5]. To our best knowledge, *this is its first application in discovering alphas in quantitative finance.*

Unlike traditional reinforcement learning tasks where mean performance is prioritized (such as game playing[11] and robotics[2]), alpha mining should focus on best-case performance. The intuitive is that *bumping into bad formulas won't hurt whereas discovering a good formula will significantly boost the overall performance.* We design a risk-seeking policy optimization to guide MCTS towards best-case scenarios, addressing the third limitation.

We extensively evaluate our framework on real-world stock data to verify our framework's superiority and validity. We compare our framework with numerous baselines under various metrics. Our method outperforms all the benchmarks in every metric. We also conduct an investment simulation study under a realistic trading setting. Our methods achieve the most profitable outcomes among all compared methods. The case study proves that the MCTS-based agent can achieve the balance between exploration and exploitation while the previous method is prone to over-exploitation. In the end, we design an ablation study to investigate the contribution of each module.

2 Related Work

Formulaic alphas. The expression space for formulaic alphas is extremely large, owing to the myriad of operands and operators. GP has been the mainstream method for discovering formulas. [8] modify the gplearn library by incorporating non-linear and time-series operators. AutoAlpha[22] enhances GP by integrating Principal Component Analysis (PCA), which aids in steering the search away from explored solution spaces. Alphaevolve [3] includes vector and matrix operator in their evolutionary framework to enhance the predictive ability of the alphas. However, complicated vectors and matrix operations lead to reduced interpretability. Alphagen [20] formulates the mining pipeline as an MDP and uses PPO to solve it.

Stock trend forecasting. Using end-to-end machine-learning models to forecast stock trends has attracted attention from researchers. The vanilla idea is to use time-series models[15, 16] to predict returns. Diffusion models are also used to model the distribution of stock returns[7]. The machine-learning methods[17, 18] usually have strong forecasting abilities but require high-quality features constructed by formulaic alphas.

3 PROBLEM FORMULATION

3.1 Formulaic Alpha

Formulaic alpha is a mathematical expression $f(\cdot)$. The formula transforms the original price data X_t of stocks and other relevant financial and market data into alpha values $z_t = f(X_t) \in \mathbb{R}^n$, in which n is the number of stocks. We use the term alpha to refer to the formula and the calculation results. The Information Coefficient (IC) is a popular metric in the industry for evaluating alphas. The IC is the Pearson correlation between the alpha value z_t and the future stock returns r_{t+1}:

$$\mathrm{IC} = \frac{\mathrm{Cov}(z_t, r_{t+1})}{\sigma_{z_t}\sigma_{r_{t+1}}}, \tag{1}$$

in which Cov is the covariance and σ is the standard deviation.

When the alpha value and the returns are ranked by their magnitude, the resulting IC is referred to as RankIC. The purpose of this approach is to emphasize the relative order of the predicted stocks. For instance, an original series of alpha values such as (0.1, 0.2, 0.3) becomes (1, 2, 3) after ranking, which is then used to calculate the RankIC:

$$\mathrm{RankIC} = \frac{\mathrm{Cov}(\mathrm{rank}(z_t), \mathrm{rank}(r_{t+1}))}{\sigma_{\mathrm{rank}(z_t)}\sigma_{\mathrm{rank}(r_{t+1})}}. \tag{2}$$

The IC, ranging from -1 to 1, with values closer to 1 or -1 indicates a stronger predictive ability of the alpha. The goal of alpha mining is to discover alpha expressions with strong predictive ability, reflected in higher IC and RankIC values. The similarity between two alphas can be assessed by calculating the correlation between them, known as the Mutual Information Coefficient (mutIC):

$$\mathrm{mutIC} = \frac{\mathrm{Cov}(z_t, z_t^*)}{\sigma_{z_t}\sigma_{z_t^*}}. \tag{3}$$

3.2 Mining Combinational Alpha

It is a common practice to use an alpha synthesis model to generate a composite alpha value from a group of alphas to achieve synergistic effects. The composite alpha can better guide the construction of the investment portfolio. Let $\mathcal{F} = \{f_1, f_2, \ldots, f_k\}$ be a set containing k alphas. The alpha synthesis model, $c(\cdot|\mathcal{F}, \omega)$, integrates the information of each alpha. Given the model parameters, ω, the composite alpha value $z_t = c(X_t|\mathcal{F}, \omega)$ can be computed.

Discovering an ideal set of alphas is a highly challenging task. An ideal alpha combination should meet the following criteria: *the alphas in the set should have high individual IC, meanwhile maintaining mutIC as low as possible with other alphas.* High mutIC between alphas indicates market information overlaps. The redundancy within the alpha pool leads to limited improvement in the composite alpha's performance.

3.3 Reverse Polish Notation

In our approach, we represent alphas using Reverse Polish Notation (RPN), setting up the stage for the subsequent design of an MDP. The RPN, derived from the post-order traversal of an expression tree, models the formula as a sequence of tokens unambiguously. With operands as leaf nodes and operators as non-leaf nodes. Figure 2 provides an example of the RPN.

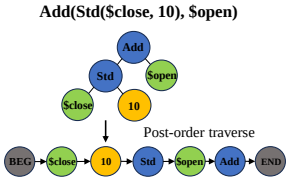


Figure 2: Reverse Polish Notation(RPN).

4 METHODOLOGY

To mine the "excellent and different" alpha set, we first formulate a reward-dense MDP that enables the search algorithm to efficiently learn the characteristics of the expression space. Then we propose a novel risk-seeking MCTS to explore the expression space. To be specific, MCTS and risk-seeking policy optimization are executed alternately to mining alphas.

4.1 Alpha Pool

The alpha pool synthesizes a group of alphas. An ideal alpha pool should be a simple and efficient machine-learning model. Given the interpretability, we choose a linear model for alpha synthesis.

We calculate the composite alpha $z_t = c(X_t|\mathcal{F}, \omega)$ through a weighted synthesis approach. Assuming there are k alphas in the pool, the model parameters $\omega = (\omega_1, \omega_2, \ldots, \omega_k)$ represent the weight coefficients for each alpha. The absolute values of the elements in the weight vector reflect the importance of the corresponding alpha. $f(X_t) = (f_1(X_t), f_2(X_t), \ldots, f_k(X_t))$ is the current value of the k alphas, and $z_t = \omega \cdot f(X_t)$ represents the value of the composite alpha. We employ Mean Squared Error (MSE) as the loss function to measure the gap between the synthesized alpha and the future stock returns:

$$\mathcal{L}(\omega) = \frac{1}{nT} \sum_{t=1}^{T} ||z_t - r_{t+1}||^2. \tag{4}$$

Using gradient descent to minimize the loss, we obtain the optimal weight ω for the current alpha set $\mathcal{F}$. The pool size K is predefined. The new alpha will be added to the pool incrementally. When a new alpha is added to the pool, gradient descent is performed to get the updated weight of the $k+1$ alphas. If the number of alphas has reached the threshold K, the alpha, with the smallest absolute weight, is removed from the pool. The pseudocode for maintaining the alpha pool is shown in Algorithm 1.

4.2 Reward-Dense MDP

We construct a reward-dense MDP for the alpha mining problem. By solving this MDP, the algorithm can identify alphas exhibiting high IC performance.

Key Concept in the MDP — Token: A Token is the fundamental building block in this MDP, representing operands and operators. The state of the MDP, s_t, is defined as the current sequence of selected Tokens, while the action, a_t, is to select the next Token. The transition is deterministic. Each decision sequence (episode) starts with a specific beginning Token (BEG) and concludes upon selecting the ending Token (END).

Design of Intermediate State Rewards: Intermediate rewards are set for states that have not reached the end state (i.e., have not selected the END Token). If the current Token sequence forms a valid RPN, we compute the IC of this unfinished sequence. If the alpha pool is non-empty, we calculate the mutIC between the sequence and each alpha in the alpha pool. The intermediate reward signal is derived from these calculations:

$$\text{Reward}_{\text{inter}} = \text{IC} - \lambda \frac{1}{k} \sum_{i=1}^{k} \text{mutIC}_i, \tag{5}$$

where k is the number of alphas in the pool, mutIC_i denotes the mutual IC value between the current alpha and the i-th alpha in the pool, and λ is a hyperparameter in the MDP.

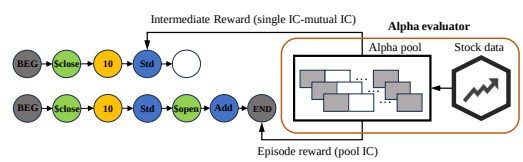


Figure 3: Reward-Dense MDP. The intermediate reward is designed for the legal but not complete expression. When the episode is terminated, an episode reward is assigned.

Episode Termination and Reward Allocation: The selection of the END Token signifies the end of an episode. At this point, the corresponding alpha is added to the alpha pool, and Algorithm 1 is executed to obtain the current composite alpha value. Ultimately, the IC of the composite alpha serves as the terminal reward $\text{Reward}_{\text{end}}$ for that episode. The maximum length of the episode is 20.

4.3 Risk-based Monte Carlo Tree Search

To effectively search the vast and discrete alpha solution space, we design a special MCTS. Figure 4 shows a single cycle of MCTS. A single cycle in MCTS consists of four phases: Selection, Expansion, Rollout, and Backpropagation. In the Selection phase, the Tree Policy is a crucial mechanism guiding the search process from the

root to the leaf node, determining which leaf node is worthy of further exploration. The Rollout phase aims to assess the value of the current leaf node by the Rollout Policy.

To enhance the efficiency of searching the formulas, this study trains a **policy network** via a novel risk-seeking policy gradient method. *The policy generated by the network is used as both the Tree Policy and Rollout Policy for the Selection and Rollout.* The training method of this network will be detailed in Section 4.4.

Each edge (s, a) in the Monte Carlo tree contains the following information: $\{N(s,a), P(s,a), Q(s,a), R(s,a)\}$, where $N(s,a)$ is the number of visits to the edge, $P(s,a)$ is the prior probability of the edge as given by the risk policy network, $Q(s,a)$ is the action value of the edge, and $R(s,a)$ is intermediate reward after the action a.

Selection and Expansion: In the Monte Carlo Tree, nodes represent the tokens of the expression, and edges represent the action of selecting the next token from the current token sequence. The root node of the tree is the BEG Token, where the search begins. During the selection process, we employ the following PUCT formula to determine the next non-leaf node to select:

$$a_t = \arg\max_a Q(s,a) + P(s,a)\frac{\sqrt{\sum_b N(s,b)}}{1+N(s,a)}. \tag{6}$$

Upon reaching a leaf node that is not an endpoint (i.e., not the END Token), we invoke the risk policy network, $p_l = \pi_{risk}(s_l)$, and assign initial values to the newly expanded edges$\{N(s,a) = 0, P(s,a) = p_l, Q(s,a) = 0, R(s,a) = 0\}$. Each time an intermediate node is selected, if the current token sequence is valid, then the reward of edge (s, a) is updated with $R(s,a) = r_t = \text{Reward}_{\text{inter}}$.

Rollout and Backpropagation: After completing the selection and reaching a non-terminal leaf node, the rollout is performed. During the rollout, the next action is sampled from the probability given by the policy network until the END Token is met. The intermediate rewards $\text{Reward}_{\text{inter}}$ generated during the rollout, along with the final reward $\text{Reward}_{\text{end}}$, are summed to get the node value estimation v_l. For non-leaf node $k = l, \ldots, 0$, we perform a $l-k$ step bootstrap and get the following cumulative rewards:

$$G_k = \sum_{i=0}^{l-1-k} \gamma^i r_{k+1+i} + v_l, \tag{7}$$

where γ is set to 1 in this scenario because we encourage the exploration of long expressions. For $k = l, \ldots, 1$, we update the data on the edge (s_{k-1}, a_k):

$$Q(s_{k-1}, a_k) = \frac{N(s_{k-1}, a_k) \times Q(s_{k-1}, a_k) + G_k}{N(s_{k-1}, a_k) + 1}, \tag{8}$$

$$N(s_{k-1}, a_k) = N(s_{k-1}, a_k) + 1. \tag{9}$$

In one search loop of MCTS, the process from the BEG Token to the END Token constitutes a complete episode trajectory

$$\tau = \{s_0, a_1, r_1, s_1, \ldots, s_{T-1}, a_T, r_T, s_T\}.$$

Episodes generated during the search process are stored in the replay buffer. When the number of episodes in the buffer reaches a predetermined threshold, the search is stopped, and these data are used to train the risk policy network for subsequent search cycles.

4.4 Risk-Seeking Policy Optimization

Mainstream RL algorithms typically aim at maximizing the expectation of cumulative rewards, which can be ineffective in optimizing the best-case performance of the policy. We design a novel policy-based RL algorithm to train a risk-seeking policy network that discovers optimal alphas by focusing on best-case scenarios. In standard planning and decision-making problems, tail risks (i.e., the worst-case performance of a policy) are often emphasized. However, in the context of alpha search, our focus shifts towards pursuing the best-case performance. We utilize quantile optimization, an effective mathematical tool for reshaping the distribution, to tailor our risk-seeking policy. *The trained risk-seeking policy serves as the rollout and tree policy in the MCTS to optimize the best-case performance during the alpha search process.*

In each episode, a trajectory τ is generated by following the action selecting policy $\pi(\cdot|\cdot;\theta)$, which is represented by a neural network with parameter θ. The corresponding cumulative reward is given by $R(\tau) = \sum_{t=0}^{T-1} \gamma^t r_t$, which follows a cumulative distribution function(CDF) $F_R(\cdot;\theta)$. Classical policy-based RL aims to optimize the expectation objective $J(\theta) = \mathbb{E}_{\tau\sim\Pi(\cdot;\theta)}[R(\tau)]$. However, our interest lies in the quantile of the cumulative reward, which can be defined as $q(\theta;\alpha) = F_R^{-1}(\alpha;\theta)$ when $F_R(\cdot;\theta)$ is continuous. We are concerned with the upper α-quantile of $R(\tau)$, thus solving the optimization problem

$$\max_{\theta\in\Theta} J_{risk}(\theta;\alpha) = \max_{\theta\in\Theta} q(\theta; 1-\alpha), \tag{10}$$

where Θ denotes the parameter space. The policy optimization procedure under the quantile criterion can be implemented by two coupled recursions. We first need to estimate the quantile of the cumulative reward. The upper α-quantile is tracked by

$$q_{i+1} = q_i + \beta(1 - \alpha - \mathbf{1}\{R(\tau_i) \le q_i\}), \tag{11}$$

which is a numerical approach for solving the root searching problem $F_R(q(1-\alpha;\theta);\theta) = 1-\alpha$. Then we can perform gradient ascent with the direction calculated by the following theorem.

Theorem 4.1. *The objective gradient $\nabla_\theta J_{risk}(\alpha;\theta)$ has the same direction as $E[D(\tau;\theta,q(\alpha;\theta))]$, where*

$$D(\tau;\theta,r) = -\mathbf{1}\{R(\tau_i) \le r\} \sum_{t=1}^{T} \nabla_\theta \log \pi(a_t|s_t;\theta). \tag{12}$$

With Theorem 4.1, we can update the network parameters by

$$\theta_{i+1} = \theta_i + \gamma D(\tau_i;\theta_i,q_i). \tag{13}$$

4.5 Training Procedure

The MCTS and the policy optimization are executed alternately to mine alphas. Trajectories sampled from the MCTS will be the training data for the risk-seeking policy network. The trained policy network will be used in the selection and rollout procedures in the MCTS. In other words, the MCTS serves as a sampler to interact with the environment; using the sample trajectories, the policy optimization works as a optimizer to train a risk-seeking policy for the MCTS sampler. The policy network is composed of a GRU feature extractor and a multilayer perceptron policy head. The pseudocode for the mining framework is shown in Algorithm 2.

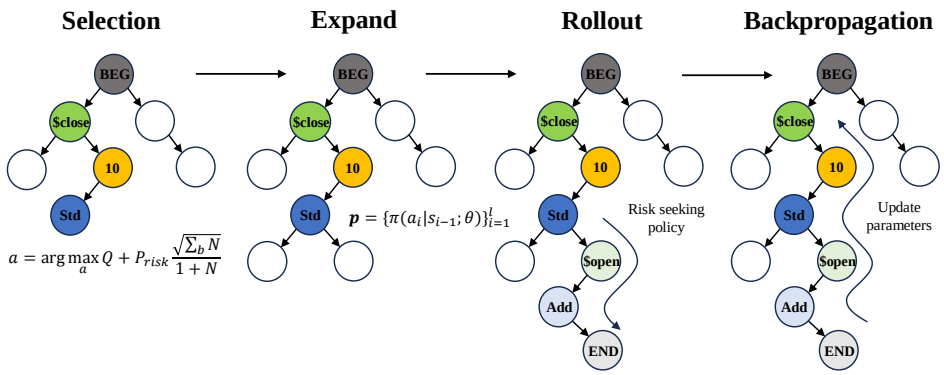


Figure 4: Search Cycle of Monte Carlo Tree Search. Selection: start from the root and reach the leaf according to the Tree Policy. Expand: expand the leaf node and initialize the newly added edges. Rollout: reach the terminated state with the Rollout Policy. Backpropagation: update the value of the edge along the trajectory with the intermediate and terminated reward.

5 Experiment

We design experiments to answer the following questions:

- **Q1:** How does our method compare with other methods?
- **Q2:** How do the synergistic alphas perform in a realistic trading scenario?
- **Q3:** Why our framework is an irreplaceable solution?
- **Q4:** How do different components in our methods contribute to the overall performance?

5.1 Experiment Setting

5.1.1 Data. We evaluate our alpha mining framework on the Chinese A-share market using two stock sets: the constituent stocks of the CSI300 and CSI500 indexes[1]. Our experiments use six features: open, high, low, close, volume, vwap(volume-weighted average price). We use the raw feature as alphas' input and mine alphas that highly correlate with future returns. The returns are computed on the close price and we have two targets for the returns. One is the **5-day** returns, the other is **10-day** returns. The dataset contains 13 years of daily data and is split into a training set (2010/01/01 to 2019/12/31), a validation set (2020/01/01 to 2020/12/31), a test set (2021/01/01 to 2022/12/31).

5.1.2 Baselines. To demonstrate the superiority of our method, we compare it with various benchmarks. First, we compare with methods about formulaic alphas.

Formulaic alphas:

- **Genetic Programming** generates alphas by performing biological operations on the expression tree. GP generates one alpha at a time using IC as the fitness measure. We use implementation by **gplearn** in the experiment.
- **Alphagen** is the current SOTA of generating formulaic alphas by PPO. We use the authors' official implementation in our experiment.

To better evaluate the effectiveness of our method, we also compare it with direct forecasting machine learning models implemented on qlib[19]. For each stock, the models look back 60 days with the 6 price/volume features to construct 360-dimension input data. The models are trained by regression on future returns and then give the predictive score. The hyperparameters of these end-to-end models are set according to the benchmarks on qlib. We compare our method with two kinds of neural network models,i.e. **MLP** and **GRU**. We also include ensemble models in the baselines, i.e. **XGBoost**[1], **LightGBM**[6], and **CatBoost**[10].

[1]The stock data is collected from https://www.baostock.com.

5.2 Main Results

To answer **Q1**, we conduct experiments on different methods including formulaic alphas generator(gplearn, alphagen) and direct stock trend forecasting model(MLP, GRU, XGBoost, LightGBM, CatBoost).

Table 1 shows our signal-based main results. Our proposed method achieves the best results across all baselines. The synergistic alphas discovered by RiskMiner not only exhibit strong predictive ability(high IC and RankIC) but also have stable performance(high ICIR). The alphagen comes in the second tier. It has competitive results on IC and RankIC. However, the gap in the ICIR indicator shows that the predictive power of the alphas generated by alphagen is not stable enough compared with our method. The end-to-end forecasting models have moderate performance in the experiment since they use raw stock data to predict future trends without elaborate feature engineering. The alphas generated by traditional GP are no longer competitive with other methods generating formulaic alpha.

Since using quantile as the risk measure to facilitate the search process, it is necessary to investigate how the model performs under different risk-seeking levels. We run the mining pipeline under quantile values of $\alpha \in \{0.6, 0.65, 0.7, 0.75, 0.8, 0.85, 0.9, 0.95\}$ and monitor the IC performance. The results are shown in Figure 5.

In the beginning, the IC scores increase as the quantile level increases. It indicates that the risk-seeking policy can efficiently discover high-quality alphas since it directly optimizes the best-case performance. However, when the quantile value exceeds 0.85, the performance of our pipeline starts to decline. It shows that an overly aggressive search policy can lead to performance degradation.

A plausible explanation for this phenomenon could be that: even though the risk-seeking policy is more likely to discover better alphas, an overly aggressive policy may stuck in some local optimum. *Mining alphas factor is different from other traditional RL tasks. We want to search as many optimums as possible instead of overly*

Table 1: Main results on CSI300 and CSI500. All experiments repeat 10 times. Values outside the parenthesis are the mean and values inside the parenthesis are the standard deviation. All the evaluation metrics are the higher the better.

Method	CSI300						CSI500					
	5 days			10 days			5 days			10 days		
	IC	ICIR	RankIC	IC	ICIR	RankIC	IC	ICIR	RankIC	IC	ICIR	RankIC
MLP	0.0273	0.1870	0.0396	0.0265	0.1772	0.0326	0.0259	0.1930	0.0389	0.0272	0.2042	0.0293
GRU	0.0383	0.2772	0.0584	0.0362	0.2883	0.0474	0.0378	0.2879	0.0576	0.0376	0.2987	0.0435
XGBoost	0.0394	0.2909	0.0448	0.0372	0.3102	0.0423	0.0417	0.3327	0.0439	0.0368	0.3576	0.0479
LightGBM	0.0403	0.4737	0.0499	0.0398	0.3342	0.0512	0.0392	0.3481	0.0409	0.0416	0.3765	0.0537
CatBoost	0.0378	0.3714	0.0467	0.0431	0.4380	0.0583	0.0427	0.4529	0.0492	0.0386	0.4237	0.0426
gplearn	0.0283	0.2425	0.0298	0.0257	0.2653	0.0372	0.0327	0.3562	0.0463	0.0254	0.2938	0.0372
	(0.0089)	(0.0247)	(0.0073)	(0.0070)	(0.0190)	(0.0082)	(0.0059)	(0.0203)	(0.0112)	(0.0063)	(0.0291)	(0.0067)
alphagen	0.0572	0.4023	0.0689	0.0553	0.3422	0.0633	0.0481	0.4296	0.0702	0.0489	0.5137	0.0678
	(0.0109)	(0.0288)	(0.0083)	(0.0090)	(0.0172)	(0.0120)	(0.0047)	(0.0260)	(0.0062)	(0.0052)	(0.0320)	(0.0065)
RiskMiner	**0.0645**	**0.5126**	**0.0734**	**0.0637**	**0.5361**	**0.0728**	**0.0596**	**0.6420**	**0.0837**	**0.0603**	**0.5920**	**0.0752**
	(0.0069)	(0.0270)	(0.0093)	(0.0083)	(0.0192)	(0.0107)	(0.0064)	(0.0183)	(0.0067)	(0.0043)	(0.0361)	(0.0055)

exploiting one local optimum. As the risk-seeking level increases, the model would spend most of the search budget on the current best local optimum and fail to investigate other possible optimums in the expression space.

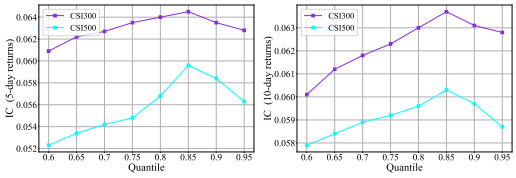


Figure 5: Model performance under different quantile level

5.3 Trading simulation

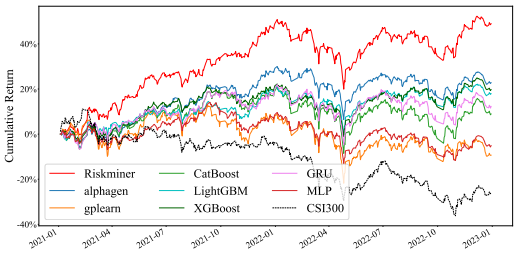


Figure 6: Trading simulation on CSI300. The lines represent the cumulative return of agents under different alpha mining methods.

To answer **Q2**, we conduct a simulated trading experiment on the test set (from 2021/01/01 to 2022/12/31). We use the 5-day alphas in the previous experiment as trading signals and we rebalance our stock position weekly(every 5 days). We adopt a simple long strategy in the experiment. Specifically, we rank the stock on day t according to the predicted score or alpha value. Then we select the top k stocks to evenly invest in them and sell currently held stocks that rank lower than k. We use cumulative returns as the portfolio metrics to evaluate the trading performance.

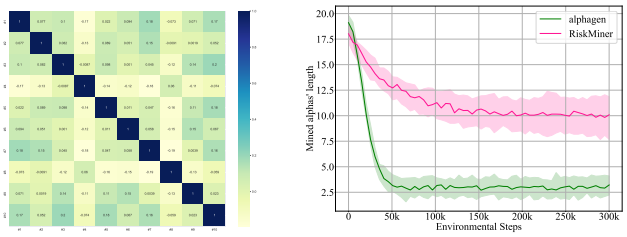
- **Cumulative return**: cumulative return is defined as the total change in the value of an investment or portfolio over a set period.

$$\mathrm{CR} = \frac{\text{Final Value}}{\text{Initial Value}} - 1$$

We set $k = 50$ in the trading experiment for all the models. The results are shown in Figure 6. Certain models, including gplearn and MLP, incur losses in the period. Other direct-forecasting benchmarks only gain limited positive cumulative returns. The alphagen has minor improvement over other benchmarks and we will elaborate on the reason in the case study. On the contrary, our proposed RiskMiner outperforms all the benchmarks by a large margin.

5.4 Case Study

We give an example of the combination of 10 alphas found by our framework in Table 2. The alpha #6 has the highest IC of 0.0365 among all the alphas. Surprisingly, the linear combination can achieve an IC as high as 0.0524 exceeding individual alphas by a large margin in the set. As shown in Figure 7(a), the 10 alphas have a low correlation with each other, indicating that each alpha contributes to the prediction in a complementary way. If the mutIC penalty coefficient λ is set to zero, the mined alphas will be too similar to generate the synergistic effect.



(a) (b)

Figure 7: (a) The correlation map of the 10 alphas. (b) The length of the alpha formula changes with the environment steps.

The prevailing method, alphagen, suffers from a severe problem: the PPO agent of alphagen will quickly converge to some short formulas after a few steps in the environment, as shown in Figure 7(b). The search space for long formulas is larger than the space for

Table 2: A 10-alpha combination.

#	Alpha	Weight	IC (CSI300)
1	$\mathrm{Kurt}(\mathrm{Rank}(close, 50), 30)$	0.0153	0.0115
2	$\lvert\mathrm{Greater}(close, 10) - 30 \cdot \mathrm{Ref}(volume, 50)\rvert$	0.0226	0.0238
3	$\mathrm{Greater}(2, vwap) - open$	0.0473	0.0325
4	$\mathrm{CSRank}(\mathrm{Med}(low, 30))$	−0.0181	−0.0231
5	$\mathrm{Kurt}(\mathrm{Corr}(\mathrm{Rank}(high, 20), vwap, 20) + \mathrm{Max}(\mathrm{Less}(open, close), 10), 5)$	0.0196	0.0217
6	$(2 + \mathrm{Ref}(\mathrm{CSRank}(\mathrm{Rank}(vwap, 50), 40))/\mathrm{Log}(high)$	0.0887	0.0365
7	$\mathrm{Std}(\mathrm{CSRank}(5/high), 40)$	0.0328	0.0105
8	$\mathrm{EMA}(\mathrm{Log}(high), 30) + \mathrm{EMA}(\mathrm{Log}(low), 10)$	−0.0283	−0.0137
9	$\mathrm{Mean}(\mathrm{Skew}(open/high, 20) + \mathrm{Sum}(volume, 5), 50)$	0.0245	0.0163
10	$\mathrm{Cov}(volume, close, 10)$	0.0311	0.0286
	Combination		0.0524

short formulas and the agent should spend more search budget on the long formulas. However, the PPO agent, overly exploiting a few local optimums, fails to fully explore the space for long formulas, and most of the search budget is wasted on short formulas. On the contrary, our framework, RiskMiner, can achieve a good balance between exploration and exploitation, maintaining the length of the searched formulas within a reasonable range. While the risk-seeking policy module enables local exploitation, the MCTS is employed for global exploration. Therefore, the combination of the two modules can find as many optimums, i.e. good formulas, as possible, answering **Q3**.

5.5 Ablation Study

Table 3: Ablation study on CSI300.

MCTS	Risk-seeking policy	5 days IC	5 days ICIR	5 days RankIC
✓	✓	**0.0645**	**0.5126**	**0.0734**
×	✓	0.0617	0.4622	0.0693
✓	×	0.0630	0.4501	0.0721

To answer **Q4**, we conduct the ablation study on the CSI300 dataset. Note that MCTS and the risk-seeking policy can work separately as alphas generators to search the expression space from the alphapool's reward. We investigate the individual performance of the two modules and the aggregate performance when they are working together. The results are shown in Table 3. The performance of individual modules degrades to a certain extent. The MCTS has a slight advantage over the Risk-seeking policy. We can infer that MCTS contributes significantly to the overall performance. By integrating with a risk-seeking policy, our methods can elevate performance to a new tier.

6 Conclusion

In this paper, we propose RiskMiner, a novel framework for generating synergistic formulaic alphas. A reward-dense MDP is designed to stabilize the search. Then we integrate MCTS with the risk-seeking policy enabling Riskminer to navigate through vast discrete solution space in alpha mining effectively with the balance between exploration and exploitation. We demonstrate the effectiveness of RiskMiner through extensive experiments on real-world datasets, showcasing its superiority and validity in discovering synergistic alphas with strong predictive abilities and stable performance. The proposed method outperforms all existing state-of-the-art methods under various metrics and realistic trading settings.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grants 72325007,72250065,72022001

A Proof of Theorem 4.1

Proof. By the definition of the α-quantile, we have

$$F_R(q(\alpha;\theta);\theta) = \alpha.$$

With the implicit function theorem, we can obtain

$$\nabla_\theta q(\alpha;\theta) = -\frac{\nabla_\theta F_R(r;\theta)}{f_R(r;\theta)}\bigg|_{r=q(\alpha;\theta)},$$

where $f_R(\cdot;\theta)$ is the non-negative density function and implies

$$\nabla_\theta q(\alpha;\theta) \propto -\nabla_\theta F_R(r;\theta)|_{r=q(\alpha;\theta)}.$$

Since $\mathbf{1}\{R(\tau) \le r\}$ is an unbiased estimator for $F_R(r;\theta)$, we derive the CDF gradient as follows:

$$\begin{aligned}\nabla_\theta F_R(r;\theta) &= \nabla_\theta \mathbb{E}[\mathbf{1}\{R(\tau) \le r\}] = \nabla_\theta \int_{\Omega_\tau} \mathbf{1}\{R(\tau) \le r\}\Pi(\tau;\theta)d\tau \\ &= \mathbb{E}[\mathbf{1}\{R(\tau) \le r\}\nabla_\theta \log \Pi(\tau;\theta)] \\ &= \mathbb{E}[\mathbf{1}\{R(\tau) \le r\}\sum_{t=1}^{T} \nabla_\theta \log \pi(a_t|s_{t-1};\theta)]\end{aligned}$$

Thus, we obtain the unbiased estimator for $-\nabla_\theta F_R(r;\theta)$ as follows:

$$D(\tau;\theta,r) = -\mathbf{1}\{R(\tau_i) \le r\}\sum_{t=1}^{T} \nabla_\theta \log \pi(a_t|s_{t-1};\theta),$$

□

B Operators and Operands

All the operands and operators used in the mining are listed in Table 4 and Table 5. There are three types of operands: price/volume feature, times deltas, and constant. The operators are divided into 2 groups: cross-section and time-series operators. In each category, operators can be further divided as unary and binary.

Table 4: operands used in our framework.

Operand	Description
Price/volume feature	*open*, *high*, *close*, *low*, *volume*, *vwap*
Times deltas	1, 5, 10, 20, 30, 40, 50
Constant	-30.0, -10.0, -5.0, -2.0, -1.0, -0.5, -0.01, 0.5, 1.0, 2.0, 5.0, 10.0, 30.0

Table 5: operators used in our framework.

Operators	Description
$\mathrm{Abs}(x)$, $\mathrm{Log}(x)$	The absolute value or natural logarithm of x.
CSRank(x)	The rank of x compared to other stocks today.
$x+y$, $x-y$, $x \cdot y$, x/y	Arithmetic operators.
$\mathrm{Greater}(x,y)$, $\mathrm{Less}(x,y)$	Comparison between two values.
$\mathrm{Ref}(x,t)$	The value of x assessed t days prior to today.
$\mathrm{Rank}(x,t)$	The rank of present x over the past t days.
$\mathrm{Skew}(x,t)$, $\mathrm{Kurt}(x,t)$	The skewness or kurtosis of the x over past t days.
$\mathrm{Mean}(x,t)$, $\mathrm{Med}(x,t)$, $\mathrm{Sum}(x,t)$	The mean, median, or sum of x over the past t days.
$\mathrm{Std}(x,t)$, $\mathrm{Var}(x,t)$	The standard deviation or variance of x over the past t days.
$\mathrm{Max}(x,t)$, $\mathrm{Min}(x,t)$	The maximum/minimum value of x over the past t days.
$\mathrm{WMA}(x,t)$, $\mathrm{EMA}(x,t)$	The weighted or exponential moving average of x over the past t days.
$\mathrm{Cov}(x,y,t)$, $\mathrm{Corr}(x,y,t)$	The covariance or correlation between x and y over the past t days.

C Pseudo Codes

we provide pseudocode for maintaining the alphapool and the overall alpha mining framework.

Algorithm 1: maintain the alpha pool

Input: alpha set $\mathcal{F} = \{f_1, f_2, \ldots, f_k\}$; a new alpha f_{new}; the alpha combination model $c(\cdot|\mathcal{F}, \omega)$
Output: optimal alpha set $\mathcal{F}^* = \{f_1^*, f_2^*, \ldots, f_k^*\}$ and its weight $\omega^* = (\omega_1^*, \omega_2^*, \ldots, \omega_k^*)$

$\mathcal{F} \leftarrow \mathcal{F} \cup f_{new}$, , $w \leftarrow w \| \mathrm{rand}()$;
for *each iteration* **do**
  Calculating $z_t = c(X_t|\mathcal{F}, \omega)$ and $\mathcal{L}(\omega)$;
  $\omega \leftarrow$ GradientDescent $(\mathcal{L}(\omega))$;
$i = \arg\min_i |\omega_i|$;
$\mathcal{F} \leftarrow \mathcal{F}/\{f_i\}$, $\omega \leftarrow (\omega_1, ..., \omega_{i-1}, \omega_{i+1}, ..., \omega_k)$;
return $\mathcal{F}$ and ω

D Implementation details

We set the alphapool size $K = 30$. The parameter λ in the MDP is set to 0.1. The GRU extractor has 4 layers and the hidden dimension is 64. The policy head is MLP with two layers of 32 neurons. In one mining iteration, the MCTS cycle will be executed 200 times, and the sampled trajectories will be used for risk-seeking policy optimization. The learning rate β and γ for quantile regression and network parameter update are 0.01 and 0.001 respectively.

Algorithm 2: alpha mining pipeline

Input: raw stock data X_t
Output: optimal alpha set $\mathcal{F}^* = \{f_1^*, f_2^*, \ldots, f_k^*\}$ and its weight $\omega^* = (\omega_1^*, \omega_2^*, \ldots, \omega_k^*)$

Initialize $\mathcal{F}$ and ω;
Initialize $\pi_{risk}(\cdot|\theta)$ and replay buffer $\mathcal{B} = \{\}$;
for *each iteration* **do**
  Reset the root node of the search tree;
  Empty $\mathcal{B}$;
  for *each interation* **do**
    Select by Equation 6 to reach the leaf node then expand;
    Rollout by $\pi_{risk}(\cdot|\theta)$ then backpropagation;
    Update $\mathcal{F}$ and ω using Algorithm 1;
    $\mathcal{B} \leftarrow \mathcal{B} \cup \tau$;
  for *each* $\tau \in \mathcal{B}$ **do**
    Using Equation 11 to estimate current quantile;
    Using Equation 13 to update policy network;
return $\mathcal{F}$ and ω

References

[1] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, August 13-17, 2016*, Balaji Krishnapuram, Mohak Shah, Alexander J. Smola, Charu C. Aggarwal, Dou Shen, and Rajeev Rastogi (Eds.). ACM, 785–794.

[2] Yuanpei Chen, Tianhao Wu, Shengjie Wang, Xidong Feng, Jiechuan Jiang, Zongqing Lu, Stephen McAleer, Hao Dong, Song-Chun Zhu, and Yaodong Yang. 2022. Towards human-level bimanual dexterous manipulation with reinforcement learning. *Advances in Neural Information Processing Systems* 35 (2022), 5150–5163.

[3] Can Cui, Wei Wang, Meihui Zhang, Gang Chen, Zhaojing Luo, and Beng Chin Ooi. 2021. AlphaEvolve: A Learning Framework to Discover Novel Alphas in Quantitative Investment. In *SIGMOD '21: International Conference on Management of Data, Virtual Event, China, June 20-25, 2021*, Guoliang Li, Zhanhuai Li, Stratos Idreos, and Divesh Srivastava (Eds.). ACM, 2208–2216.

[4] Yitong Duan, Lei Wang, Qizhong Zhang, and Jian Li. 2022. Factorvae: A probabilistic dynamic factor model based on variational autoencoder for predicting cross-sectional stock returns. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 4468–4476.

[5] Alhussein Fawzi, Matej Balog, Aja Huang, Thomas Hubert, Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Francisco J R Ruiz, Julian Schrittwieser, Grzegorz Swirszcz, et al. 2022. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature* 610, 7930 (2022), 47–53.

[6] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett (Eds.). 3146–3154.

[7] Kelvin JL Koa, Yunshan Ma, Ritchie Ng, and Tat-Seng Chua. 2023. Diffusion Variational Autoencoder for Tackling Stochasticity in Multi-Step Regression Stock Price Prediction. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 1087–1096.

[8] Xiaoming Lin, Ye Chen, Ziyu Li, and Kang He. 2019. *Stock Alpha Mining Based On Genetic Algorithm*. Technical Report. Huatai Securities Research Center. https://crm.htsc.com.cn/doc/2019/10750101/f75b4b6a-2bdd-4694-b696-4c62528791ea.pdf

[9] Daniel J Mankowitz, Andrea Michi, Anton Zhernov, Marco Gelmi, Marco Selvi, Cosmin Paduraru, Edouard Leurent, Shariq Iqbal, Jean-Baptiste Lespiau, Alex Ahern, et al. 2023. Faster sorting algorithms discovered using deep reinforcement learning. *Nature* 618, 7964 (2023), 257–263.

[10] Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. 2018. CatBoost: unbiased boosting with categorical features. *Advances in neural information processing systems* 31 (2018).

[11] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, et al. 2020. Mastering atari, go, chess and shogi by planning with a learned model. *Nature* 588, 7839 (2020), 604–609.

[12] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. (2017). arXiv:1707.06347

[13] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. 2018. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science* 362, 6419 (2018), 1140–1144.

[14] I. Tulchinsky. 2015. *Finding Alphas: A Quantitative Approach to Building Trading Strategies*. 1–253 pages.

[15] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[16] Heyuan Wang, Tengjiao Wang, Shun Li, Jiayi Zheng, Shijie Guan, and Wei Chen. 2022. Adaptive long-short pattern transformer for stock investment selection. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence*. 3970–3977.

[17] Wentao Xu, Weiqing Liu, Lewen Wang, Yingce Xia, Jiang Bian, Jian Yin, and Tie-Yan Liu. 2021. HIST: A Graph-based Framework for Stock Trend Forecasting via Mining Concept-Oriented Shared Information. (2021). arXiv:2110.13716

[18] Mengyuan Yang, Mengying Zhu, Qianqiao Liang, Xiaolin Zheng, and MengHan Wang. 2023. Spotlight news driven quantitative trading based on trajectory optimization. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*. 4930–4939.

[19] Xiao Yang, Weiqing Liu, Dong Zhou, Jiang Bian, and Tie-Yan Liu. 2020. Qlib: An AI-oriented Quantitative Investment Platform. (2020). arXiv:2009.11189

[20] Shuo Yu, Hongyan Xue, Xiang Ao, Feiyang Pan, Jia He, Dandan Tu, and Qing He. 2023. Generating Synergistic Formulaic Alpha Collections via Reinforcement Learning. *arXiv preprint arXiv:2306.12964* (2023).

[21] Liheng Zhang, Charu C. Aggarwal, and Guo-Jun Qi. 2017. Stock Price Prediction via Discovering Multi-Frequency Trading Patterns. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Halifax, NS, Canada, August 13 - 17, 2017*. ACM, 2141–2149.

[22] Tianping Zhang, Yuanqi Li, Yifei Jin, and Jian Li. 2020. AutoAlpha: an Efficient Hierarchical Evolutionary Algorithm for Mining Alpha Factors in Quantitative Investment. *arXiv preprint arXiv:2002.08245* (2020).

[23] Lifan Zhao, Shuming Kong, and Yanyan Shen. 2023. DoubleAdapt: A Meta-learning Approach to Incremental Learning for Stock Trend Forecasting. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3492–3503.